

PROJETO DO DESAFIO DO HACKATHON COBOL 2026

SISTEMA FINANCEIRO — FINANCE CORE

Bootcamp Hackathon COBOL 2026
Projeto Final: SISTEMA FINANCEIRO - FINANCE CORE
Tipo de execução: COBOL Batch
Formato: Arquivos Sequenciais LINE SEQUENTIAL
Integração moderna simulada: JSON manual

1. Visão Geral

O desafio do Hackathon consiste no desenvolvimento de um sistema financeiro em COBOL, simulando um backend corporativo chamado:

SISTEMA FINANCEIRO - FINANCE CORE

O sistema deverá processar clientes e transações bancárias a partir de arquivos sequenciais, aplicando regras de negócio, atualizando saldos, gerando extratos, registrando rejeições e produzindo um arquivo JSON final para simular integração com sistemas modernos.

O desafio será composto por **2 (dois) programas obrigatórios**:

BHCPH000 - Gerador da massa oficial de entrada
BHCPH001 - Sistema Financeiro - Finance Core

2. Programas Obrigatórios

2.1. Programa BHCPH000

Programa: BHCPH000
Nome....: GERADOR DE MASSA - FINANCE CORE
Objetivo: Gerar os arquivos oficiais de entrada do desafio.
Execução: COBOL Batch

O programa BHCPH000 deverá gerar:

BHCFHCLI - Arquivo de clientes, com 20 registros
BHCFHTRA - Arquivo de transações, com 40 registros

2.2. Programa BHCPH001

Programa: BHCPH001
Nome....: SISTEMA FINANCEIRO - FINANCE CORE
Objetivo: Processar clientes e transações financeiras.
Execução: COBOL Batch

O programa BHCPH001 deverá ler os arquivos gerados pelo BHCPH000:

BHCFHCLI
BHCFHTRA

E gerar:

BHCFHEXT - Extrato de transações válidas
BHCFHLOG - Log de transações rejeitadas
BHCFHJSN - JSON final

3. Fluxo Oficial do Desafio

1. Desenvolver o programa BHCPH000.
2. Executar o BHCPH000.
3. Gerar os arquivos BHCFHCLI e BHCFHTRA.
4. Desenvolver o programa BHCPH001.
5. Executar o BHCPH001.
6. Ler os arquivos gerados pelo BHCPH000.
7. Processar clientes e transações.
8. Gerar extrato, log e JSON.
9. Exibir totalizadores na SYSOUT.
10. Entregar fontes, arquivos gerados e evidências.

Fluxo visual:

BHCPH000

```
|  
|-- gera --> BHCFHCLI  
|-- gera --> BHCFHTRA  
|  
v  
BHCPH001  
|  
|-- gera --> BHCFHEXT  
|-- gera --> BHCFHLOG  
|-- gera --> BHCFHJSN
```

4. Objetivo Geral do Projeto

Desenvolver uma solução COBOL Batch capaz de:

1. Gerar arquivos oficiais de entrada.
2. Ler cadastro de clientes.
3. Ler transações financeiras.
4. Validar movimentações.
5. Atualizar saldos.
6. Gerar extrato de transações válidas.
7. Gerar log de rejeições.
8. Gerar arquivo JSON final.
9. Exibir totalizadores.

10. Tratar FILE STATUS.
11. Seguir padrões profissionais de codificação COBOL.

5. Conteúdos Aplicados

O desafio avaliará os seguintes conteúdos:

IDENTIFICATION DIVISION
ENVIRONMENT DIVISION
DATA DIVISION
PROCEDURE DIVISION
FILE SECTION
WORKING-STORAGE SECTION
LOCAL-STORAGE SECTION
Variáveis
PIC
MOVE
ADD
SUBTRACT
COMPUTE
IF
EVALUATE
PERFORM
PERFORM UNTIL
PERFORM VARYING
OCCURS
FILLER
REDEFINES, opcional
Nível 88, recomendado
CALL, recomendado/bônus
LINKAGE SECTION, se houver subprograma
OPEN
READ INTO
WRITE FROM
CLOSE
FILE STATUS
Totalizadores
Log de rejeições
STRING
JSON manual

6. Conteúdos Não Obrigatórios

Os temas abaixo **não serão obrigatórios como implementação real**:

JCL
DB2
CICS

Eles poderão aparecer apenas como contexto conceitual, documentação ou simulação.

7. Padrão de Nomenclatura

7.1. Programas Principais

Máscara:

BHCPH*

Onde:

Parte	Significado
BHC	Bootcamp Hackathon COBOL
P	Programa principal
H	Hackathon
*	Número sequencial

Exemplos:

BHCPH000

BHCPH001

7.2. Subprogramas

Máscara:

BHCSH*

Onde:

Parte	Significado
BHC	Bootcamp Hackathon COBOL
S	Subprograma/Subrotina
H	Hackathon
*	Número sequencial

Exemplos:

BHCSH001

BHCSH002

7.3. Arquivos

Máscara:

BHCFH*

Onde:

Parte	Significado
BHC	Bootcamp Hackathon COBOL
F	File / Arquivo
H	Hackathon
*	Identificação funcional

Arquivos oficiais:

Arquivo	Função
BHCFHCLI	Clientes
BHCFHTRA	Transações
BHCFHEXT	Extrato
BHCFHLOG	Log
BHCFHJSN	JSON

8. Padrão de Arquivos

Todos os arquivos deverão usar:

ORGANIZATION IS **LINE** SEQUENTIAL

Motivo:

O ambiente local OpenCOBOL trabalha melhor com LINE SEQUENTIAL para arquivos texto, garantindo quebra de linha visível a cada WRITE.

9. Arquivos do Projeto

Arquivo	Programa	Tipo	Quantidade	Descrição
BHCFHCLI	BHCPH000	Saída	20	Clientes
BHCFHTRA	BHCPH000	Saída	40	Transações
BHCFHCLI	BHCPH001	Entrada	20	Clientes
BHCFHTRA	BHCPH001	Entrada	40	Transações

BHCFHEXT	BHCPH001	Saída	Variável	Extrato
BHCFHLOG	BHCPH001	Saída	Variável	Log
BHCFHJSN	BHCPH001	Saída	Variável	JSON

10. Padrão de Declaração de Arquivos

10.1. Regra Geral

Na FILE SECTION, deverá ser declarado **apenas o registro físico** do arquivo, usando PIC x com o tamanho total do registro.

O nome do registro físico deverá seguir o padrão:

```
NOME-DO-ARQUIVO-REG
```

Exemplos:

```
BHCFHCLI-REG
BHCFHTRA-REG
BHCFHEXT-REG
BHCFHLOG-REG
BHCFHJSN-REG
```

Na LOCAL-STORAGE SECTION, deverão ser declarados os layouts detalhados dos arquivos, utilizando prefixo:

```
FD-
```

Exemplos:

```
FD-REG-CLI
FD-REG-TRA
FD-REG-EXT
FD-REG-LOG
FD-REG-JSN
```

10.2. Padrão de Leitura

As leituras deverão utilizar:

```
READ FD-ARQ-XXX INTO FD-REG-XXX
```

Exemplo:

```
READ FD-ARQ-CLI INTO FD-REG-CLI
READ FD-ARQ-TRA INTO FD-REG-TRA
```

10.3. Padrão de Gravação

As gravações deverão utilizar:

```
WRITE NOME-DO-ARQUIVO-REG FROM FD-REG-XXX
```

Exemplo:

```
WRITE BHCFHCLI-REG FROM FD-REG-CLI  
WRITE BHCFHTRA-REG FROM FD-REG-TRA  
WRITE BHCFHEXT-REG FROM FD-REG-EXT  
WRITE BHCFHLOG-REG FROM FD-REG-LOG  
WRITE BHCFHJSN-REG FROM FD-REG-JSN
```

10.4. Tabela de Registros

Arquivo	Nome lógico	Registro físico FILE SECTION	Layout LOCAL- STORAGE	Tamanho
Clientes	FD-ARQ-CLI	BHCFHCLI-REG	FD-REG-CLI	58
Transações	FD-ARQ-TRA	BHCFHTRA-REG	FD-REG-TRA	31
Extrato	FD-ARQ-EXT	BHCFHEXT-REG	FD-REG-EXT	59
Log	FD-ARQ-LOG	BHCFHLOG-REG	FD-REG-LOG	91
JSON	FD-ARQ-JSN	BHCFHJSN-REG	FD-REG-JSN	200

11. Prefixos Obrigatórios de Variáveis

Tipo	Prefixo/Padrão
Controle	CTE-
File Status	FS-
Arquivo lógico	FD-ARQ-*
Registro físico FILE SECTION	BHCFH*-REG
Layout de arquivo LOCAL-STORAGE	FD-REG-*
Campos de arquivo	FD-*
Working-Storage	WS-
Local-Storage auxiliar	GDA-
Linkage Section	LK-

Índices	WS-IDX- ou IDX-
Totalizadores	WS-TOT-

Observação:

O prefixo FD- será usado para layouts e campos de arquivo, mesmo quando declarados na LOCAL-STORAGE SECTION.

12. Limites de Nomenclatura

Programas.....: máximo 8 caracteres
 Subprogramas.....: máximo 8 caracteres
 Arquivos.....: máximo 8 caracteres
 Variáveis/constantes....: máximo 30 caracteres
 SECTIONS.....: máximo 30 caracteres

13. Lista Oficial de Abreviações

Termo	Abreviação
ARQUIVO	ARQ
ATUAL	ATU
ANTERIOR	ANT
ATIVO	ATV
BLOQUEADO	BLOQ
CADASTRO	CAD
CÓDIGO	CD
CLIENTE	CLI
CONTA	CTA
CONTADOR	CONT
CONTROLE	CTRL
CORRENTE	CORR
DATA	DT
DEPÓSITO	DEP
DESCRIÇÃO	DESC

DETALHE	DET
ENTRADA	ENT
ERRO	ERR
EXECUÇÃO	EXEC
EXTRATO	EXT
FINAL	FIM
GERAR	GER
GRAVAR	GRV
HISTÓRICO	HIST
HORA	HR
IDENTIFICAÇÃO	ID
INATIVO	INAT
INICIAL	INI
LANÇAMENTO	LCTO
LEITURA	LEIT
MENSAGEM	MSG
MOVIMENTO	MOV
NOME	NM
NÚMERO	NR
OPERAÇÃO	OPER
PAGAMENTO	PAG
PROCESSAMENTO	PROC
QUANTIDADE	QT
REGISTRO	REG
REJEITADO	REJ
RETORNO	RET
SAÍDA	SAI
SALDO	SDO

SEQUENCIAL	SEQ
STATUS	ST
TOTAL	TOT
TRANSAÇÃO	TRA
TRANSFERÊNCIA	TRF
VALIDAÇÃO	VALID
VALOR	VLR

14. Padrão Visual das DIVISIONs

Todas as DIVISIONs deverão ser cercadas por linhas de asteriscos.

<pre> ***** IDENTIFICATION DIVISION. ***** ***** ENVIRONMENT DIVISION. ***** ***** DATA DIVISION. ***** ***** PROCEDURE DIVISION. ***** </pre>

15. Padrão Visual das SECTIONs

Todas as SECTIONs deverão ser cercadas por linhas tracejadas.

<pre> *----- CONFIGURATION SECTION. *----- </pre>
--

Exemplos:

<pre> *----- INPUT-OUTPUT SECTION. *----- *----- FILE SECTION. </pre>
--

```

*-----

*-----
WORKING-STORAGE          SECTION.
*-----

*-----
LOCAL-STORAGE           SECTION.
*-----

*-----
100000-INICIALIZAR      SECTION.
*-----

```

16. Coluna da Palavra SECTION

A palavra SECTION deverá iniciar na **coluna 40**.

Exemplo:

100000-INICIALIZAR	SECTION.
--------------------	----------

17. Alinhamento na PROCEDURE DIVISION

Na PROCEDURE DIVISION, os termos abaixo deverão ser alinhados preferencialmente a partir da **coluna 40**, sempre que possível:

TO
FROM
INTO
USING
GIVING
VALUE

Exemplos:

MOVE ZEROS	TO WS-TOT-CLI-LIDOS
MOVE SPACES	TO WS-MSG-ERR
ADD 1	TO WS-TOT-TRA-LIDAS
SUBTRACT WS-VLR-TRA	FROM WS-SDO-ATU
READ FD-ARQ-TRA	INTO FD-REG-TRA
WRITE BHCFHEXT-REG	FROM FD-REG-EXT
CALL 'BHCSH001'	USING LK-AREA-VALID

Observação:

Em comandos longos, o alinhamento poderá ser flexibilizado para preservar legibilidade e compilação correta.

18. Padrão de Encerramento das SECTIONS

Toda SECTION da PROCEDURE DIVISION deverá ser encerrada com:

```
.  
NNNNNN-FIM.  
EXIT.
```

Regras obrigatórias

1. O ponto deve estar isolado em uma linha.
2. O ponto deve estar alinhado na coluna 12.
3. O ponto deve vir antes do parágrafo NNNNNN-FIM.
4. O parágrafo NNNNNN-FIM deve possuir EXIT.
5. Esse padrão é obrigatório por regra da empresa/escola.

Exemplo correto:

```
*-----  
100000-INICIALIZAR          SECTION.  
*-----  
  
    PERFORM 110000-INICIALIZAR-VARS  
    PERFORM 120000-BUSCAR-DT-HR  
    PERFORM 130000-EXIBIR-CAB  
    PERFORM 140000-ABRIR-ARQS  
  
.  
100000-FIM.  
EXIT.
```

19. IDENTIFICATION DIVISION — BHCPH000

```
*****  
* SIGLA.....: BHC - BOOTCAMP HACKATHON COBOL  
* PROGRAMA...: BHCPH000  
* ANALISTA...: HILARIO  
* AUTOR.....: SEU NOME  
* DATA.....: 04/07/2026  
* OBJETIVO...: GERADOR DE MASSA - FINANCE CORE  
* EXECUCAO...: COBOL - BATCH  
* -----  
* VRS DATA      RESPONSABEL      DESCRICAO DA VERSAO  
* ---  
* 001 04.07.26 SEU NOME          IMPLANTACAO  
* -----  
*****  
  
*****  
IDENTIFICATION DIVISION.
```

```
*****

PROGRAM-ID. BHCPH000.

*****
```

20. IDENTIFICATION DIVISION — BHCPH001

```
*****
* SIGLA.....: BHC - BOOTCAMP HACKATHON COBOL
* PROGRAMA..: BHCPH001
* ANALISTA..: HILARIO
* AUTOR.....: SEU NOME
* DATA.....: 04/07/2026
* OBJETIVO..: SISTEMA FINANCEIRO - FINANCE CORE
* EXECUCAO..: COBOL - BATCH
* -----
* VRS DATA      RESPONSVEL      DESCRICAO DA VERSAO
* ---
* 001 04.07.26 SEU NOME          IMPLANTACAO
* -----
*****

*****
IDENTIFICATION DIVISION.
*****

PROGRAM-ID. BHCPH001.

*****
```

21. Especificação do Programa BHCPH000

21.1. Objetivo

O programa BHCPH000 deverá gerar os arquivos oficiais de entrada do desafio.

Arquivos gerados:

```
BHCFHCLI - 20 registros de clientes
BHCFHTRA - 40 registros de transações
```

21.2. Requisitos obrigatórios do BHCPH000

1. Seguir o padrão visual do Hackathon.
2. Usar ORGANIZATION IS LINE SEQUENTIAL.
3. Declarar FILE STATUS para os arquivos.
4. Abrir arquivos com OPEN OUTPUT.
5. Gravar exatamente 20 registros em BHCFHCLI.

6. Gravar exatamente 40 registros em BHCFHTRA.
7. Usar WRITE FROM.
8. Fechar os arquivos.
9. Exibir totalizadores na SYSOUT.
10. Finalizar com GOBACK.

21.3. Estrutura Sugerida do BHCPH000

Seguir padrões definidos.

22. Especificação do Programa BHCPH001

22.1. Objetivo

O programa BHCPH001 deverá processar clientes e transações financeiras.

Entradas:

BHCFHCLI
BHCFHTRA

Saídas:

BHCFHEXT
BHCFHLOG
BHCFHJSN

22.2. Requisitos obrigatórios do BHCPH001

1. Ler 20 clientes do arquivo BHCFHCLI.
2. Carregar clientes em tabela interna com OCCURS.
3. Ler 40 transações do arquivo BHCFHTRA.
4. Validar cada transação.
5. Atualizar saldo quando a transação for válida.
6. Gerar extrato para transações válidas.
7. Gerar log para transações rejeitadas.
8. Gerar JSON final manualmente.
9. Exibir totalizadores na SYSOUT.
10. Tratar FILE STATUS.
11. Finalizar com GOBACK.

22.3. Estrutura Sugerida do BHCPH001

Seguir padrões definidos.

23. Layout do Arquivo BHCFHCLI

Arquivo:

BHCFHCLI

Quantidade:

20 registros

Tamanho:

58 posições

Layout detalhado na LOCAL-STORAGE SECTION:

```
01  FD-REG-CLI.  
    05  FD-CLI-CD          PIC 9(005).  
    05  FD-CLI-NM          PIC X(030).  
    05  FD-CLI-CPF          PIC 9(011).  
    05  FD-CLI-SD0          PIC 9(009)V99.  
    05  FD-CLI-ST          PIC X(001).
```

Status:

Código	Significado
A	Ativo
I	Inativo
B	Bloqueado

24. Layout do Arquivo BHCFHTRA

Arquivo:

BHCFHTRA

Quantidade:

40 registros

Tamanho:

31 posições

Layout detalhado na LOCAL-STORAGE SECTION:

```
01  FD-REG-TRA.  
    05  FD-TRA-ID          PIC 9(006).
```

05 FD-TRA-CD-CLI	PIC 9(005).
05 FD-TRA-TP	PIC X(001).
05 FD-TRA-VLR	PIC 9(009)V99.
05 FD-TRA-DT	PIC 9(008).

Tipos válidos:

Tipo	Descrição	Regra
D	Depósito	Soma ao saldo
S	Saque	Subtrai do saldo
P	Pagamento	Subtrai do saldo

25. Layout do Arquivo BHCFHEXT

Arquivo:

BHCFHEXT

Tamanho:

59 posições

Layout detalhado:

01	FD-REG-EXT.	
05	FD-EXT-ID-TRA	PIC 9(006).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-CD-CLI	PIC 9(005).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-TP-TRA	PIC X(001).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-VLR-TRA	PIC 9(009)V99.
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-SDO-ANT	PIC 9(009)V99.
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-SDO-ATU	PIC 9(009)V99.
05	FILLER	PIC X(001) VALUE ';'.
05	FD-EXT-DT-TRA	PIC 9(008).

26. Layout do Arquivo BHCFHLOG

Arquivo:

BHCFHLOG

Tamanho:

91 posições

Layout detalhado:

01	FD-REG-LOG.	
05	FD-LOG-ID-TRA	PIC 9(006).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-LOG-CD-CLI	PIC 9(005).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-LOG-TP-TRA	PIC X(001).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-LOG-VLR-TRA	PIC 9(009)V99.
05	FILLER	PIC X(001) VALUE ';'.
05	FD-LOG-CD-ERR	PIC X(002).
05	FILLER	PIC X(001) VALUE ';'.
05	FD-LOG-MSG-ERR	PIC X(060).

27. Layout do Arquivo BHCFHJSN

Arquivo:

BHCFHJSN

Tamanho sugerido de linha:

200 posições

Layout:

01	FD-REG-JSN	PIC X(200).
----	------------	-------------

Extensão sugerida no ambiente local:

BHCFHJSN.json

28. Massa Oficial — BHCFHCLI

O programa BHCPH000 deverá gerar exatamente os 20 registros abaixo no arquivo BHCFHCLI.

00001	JOAO SILVA	1234567890100000150000A
00002	MARIA SOUZA	2345678901200000087550A
00003	CARLOS LIMA	3456789012300000005000A
00004	ANA PEREIRA	4567890123400000250000A
00005	PAULO SANTOS	5678901234500000000000A
00006	FERNANDA COSTA	6789012345600000032000A
00007	RICARDO ALMEIDA	7890123456700000120000A

00008JULIANA ROCHA	8901234567800000065000A
00009MARCOS OLIVEIRA	9012345678900000007500A
00010PATRICIA GOMES	0123456789000000180000A
00011LUCAS MARTINS	1122334455600000040000A
00012CAMILA RIBEIRO	2233445566700000099000A
00013ROBERTO BARBOSA	3344556677800000003000A
00014BEATRIZ MENDES	4455667788900000145000A
00015GUSTAVO NUNES	5566778899000000025000A
00016HELENA DIAS	6677889900100000070000I
00017RAFAEL FREITAS	7788990011200000110000I
00018LARISSA CAMPOS	8899001122300000055000I
00019EDUARDO MOREIRA	9900112233400000200000B
00020TATIANE AZEVEDO	0011223344500000033000B

Observação:

O saldo possui 11 dígitos, com 2 casas decimais implícitas.
Exemplo: 00000150000 representa 1.500,00.

29. Massa Oficial — BHCFHTRA

O programa BHCPH000 deverá gerar exatamente os 40 registros abaixo no arquivo BHCFHTRA.

00000100001D00000005000020260704
00000200002S00000001000020260704
00000300003P00000000200020260704
00000400004S00000003000020260704
00000500005D00000002500020260704
00000600006P00000001000020260704
00000700007S00000005000020260704
00000800008D00000007500020260704
00000900009P00000000250020260704
00001000010S00000002000020260704
00001100011D00000001000020260704
00001200012P00000000500020260704
00001300013S00000000200020260704
00001400014D00000003000020260704
00001500015P00000001000020260704
00001600001S00000002500020260704
00001700002D00000004000020260704
00001800003S00000001000020260704
00001900004P00000005000020260704
00002000005S00000001000020260704
00002199999D00000001000020260704
00002288888S00000000500020260704
00002377777P00000000750020260704
00002466666D00000002000020260704
00002555555S00000000100020260704
00002600016D00000001000020260704
00002700017S00000000500020260704
00002800018P00000000250020260704
00002900019D00000005000020260704
00003000020S00000001000020260704

00003100001X0000001000020260704
00003200002Z0000001000020260704
00003300003T0000001000020260704
00003400004S0000003000020260704
00003500006P0000001000020260704
00003600009S0000001000020260704
00003700013P0000005000020260704
00003800005D0000000000020260704
00003900007S0000000000020260704
00004000008P0000000000020260704

30. Regras de Negócio do BHCPH001

30.1. Cliente inexistente

Se o cliente da transação não existir no cadastro:

Rejeitar transação
Gerar log
Não alterar saldo

Código de erro:

01 - CLIENTE NAO ENCONTRADO

30.2. Cliente inativo

Se o cliente estiver com status I:

Rejeitar transação
Gerar log
Não alterar saldo

Código de erro:

02 - CLIENTE INATIVO

30.3. Cliente bloqueado

Se o cliente estiver com status B:

Rejeitar transação
Gerar log
Não alterar saldo

Código de erro:

03 - CLIENTE BLOQUEADO

30.4. Tipo de transação inválido

Somente são válidos:

D - Depósito
S - Saque
P - Pagamento

Qualquer outro tipo deverá ser rejeitado.

Código de erro:

04 - TIPO DE TRANSACAO INVALIDO

30.5. Valor inválido

Se o valor da transação for igual a zero:

Rejeitar transação
Gerar log
Não alterar saldo

Código de erro:

05 - VALOR INVALIDO

30.6. Saldo insuficiente

Para saque e pagamento, o cliente deverá possuir saldo suficiente.

Se não possuir:

Rejeitar transação
Gerar log
Não alterar saldo

Código de erro:

06 - SALDO INSUFICIENTE

30.7. Depósito

Para transação tipo D:

```
Saldo atual = saldo atual + valor da transação
```

30.8. Saque

Para transação tipo S:

```
Saldo atual = saldo atual - valor da transação
```

Desde que exista saldo suficiente.

30.9. Pagamento

Para transação tipo P:

```
Saldo atual = saldo atual - valor da transação
```

Desde que exista saldo suficiente.

31. Ordem Recomendada de Validação

Para cada transação, recomenda-se validar nesta ordem:

1. Cliente existe?
2. Cliente está ativo?
3. Tipo de transação é válido?
4. Valor da transação é maior que zero?
5. Existe saldo suficiente, quando for saque ou pagamento?

A primeira inconsistência encontrada deverá gerar a rejeição da transação.

32. Tabela Interna de Clientes

O BHCPH001 deverá armazenar os clientes em memória usando OCCURS.

Exemplo:

```
01 WS-TB-CLI.  
    05 WS-CLI OCCURS 100 TIMES.  
        10 WS-CLI-CD          PIC 9(005).  
        10 WS-CLI-NM          PIC X(030).  
        10 WS-CLI-CPF          PIC 9(011).  
        10 WS-CLI-SDO          PIC 9(009)V99.  
        10 WS-CLI-ST           PIC X(001).
```

Embora o arquivo oficial tenha 20 clientes, a tabela poderá suportar até 100 posições.

33. Geração do Extrato

Para toda transação válida, o programa deverá gerar um registro no arquivo:

```
BHCFHEXT
```

O extrato deverá conter:

```
ID da transação
Código do cliente
Tipo da transação
Valor da transação
Saldo anterior
Saldo atual
Data da transação
```

A gravação deverá utilizar:

```
WRITE BHCFHEXT-REG FROM FD-REG-EXT
```

34. Geração do Log

Para toda transação rejeitada, o programa deverá gerar um registro no arquivo:

```
BHCFHLOG
```

O log deverá conter:

```
ID da transação
Código do cliente
Tipo da transação
Valor da transação
Código do erro
Mensagem do erro
```

A gravação deverá utilizar:

```
WRITE BHCFHLOG-REG FROM FD-REG-LOG
```

35. Geração do JSON Final

Ao final do processamento, o programa deverá gerar o arquivo:

```
BHCFHJSN
```

Extensão sugerida:

BHCFHJSN.json

O JSON deverá conter, no mínimo:

Nome do sistema
Programa executor
Totalizadores
Lista de clientes com saldo final

Exemplo conceitual:

jsonCopy

```
{
  "sistema": "FINANCE CORE",
  "programa": "BHCPH001",
  "totalizadores": {
    "clientesLidos": 20,
    "transacoesLidas": 40,
    "transacoesValidas": 25,
    "transacoesRejeitadas": 15
  },
  "clientes": [
    {
      "codigo": "00001",
      "nome": "JOAO SILVA",
      "saldoFinal": "00000175000",
      "status": "A"
    }
  ]
}
```

Regras para o JSON:

1. Deve ser gerado manualmente.
2. Deve usar STRING, MOVE e WRITE FROM.
3. Deve controlar vírgulas entre objetos.
4. Não deve deixar vírgula após o último objeto.
5. Campos em branco devem ser representados como "".
6. O arquivo deve ser legível como JSON.

36. FILE STATUS

Todos os arquivos deverão possuir variável de FILE STATUS.

Exemplo:

```
SELECT FD-ARQ-CLI ASSIGN TO 'BHCFLI.txt'
      ORGANIZATION IS LINE SEQUENTIAL
      FILE STATUS IS FS-CLI.
```

36.1. Códigos de FILE STATUS a Tratar

FILE STATUS	Significado
00	Operação realizada com sucesso
10	Fim de arquivo
35	Arquivo não encontrado
37	Modo de abertura incompatível
39	Atributos incompatíveis
41	Arquivo já aberto
42	Arquivo não aberto
46	READ sem registro válido
47	READ não permitido
48	WRITE não permitido
49	REWRITE/DELETE não permitido
OTHER	Erro não mapeado

36.2. SECTION Recomendada

Criar uma SECTION para tratamento de FILE STATUS:

```
*-----
      810000-VALIDAR-FS          SECTION.
*-----
```

Essa rotina deverá usar EVALUATE.

37. Tratamento de Erro Técnico

Criar uma SECTION para erro técnico:

```
*-----
      800000-TRATAR-ERRO        SECTION.
*-----
```

Ela deverá exibir, no mínimo:

Programa
Arquivo
Operação
FILE STATUS
Mensagem de erro

38. Totalizadores Obrigatórios

38.1. Totalizadores do BHCPH000

O BHCPH000 deverá exibir:

Total de clientes gravados
Total de transações gravadas

Sugestão:

01 WS-TOT-CLI-GRV	PIC 9(005) VALUE ZEROS.
01 WS-TOT-TRA-GRV	PIC 9(005) VALUE ZEROS.

38.2. Totalizadores do BHCPH001

O BHCPH001 deverá exibir:

Total de clientes lidos
Total de transações lidas
Total de transações válidas
Total de transações rejeitadas
Total de registros gravados no extrato
Total de registros gravados no log
Total de linhas gravadas no JSON

Sugestão:

01 WS-TOTALIZADORES.	
05 WS-TOT-CLI-LIDOS	PIC 9(005) VALUE ZEROS.
05 WS-TOT-TRA-LIDAS	PIC 9(005) VALUE ZEROS.
05 WS-TOT-TRA-VALIDAS	PIC 9(005) VALUE ZEROS.
05 WS-TOT-TRA-REJ	PIC 9(005) VALUE ZEROS.
05 WS-TOT-EXT-GRV	PIC 9(005) VALUE ZEROS.
05 WS-TOT-LOG-GRV	PIC 9(005) VALUE ZEROS.
05 WS-TOT-JSN-GRV	PIC 9(005) VALUE ZEROS.

39. Cabeçalho Obrigatório na SYSOUT

Cada programa deverá exibir cabeçalho na SYSOUT.

Exemplo para BHCPH001:

```

*****
* SIGLA.....: BHC - BOOTCAMP HACKATHON COBOL
* PROGRAMA...: BHCPH001
* ANALISTA...: HILARIO
* AUTOR.....: SEU NOME
* DATA/HORA.: NN/07/2026 - NN:NN:NN
* OBJETIVO...: SISTEMA FINANCEIRO - FINANCE CORE
* EXECUCAO...: COBOL - BATCH
*****

```

40. Uso de COPYBOOK

O uso de COPYBOOK será permitido.

Sugestões:

Copybook	Conteúdo
BHCCLIEN	Layout de cliente
BHCTRAXX	Layout de transação
BHCEXTXX	Layout de extrato
BHCLOGXX	Layout de log
BHCPARAM	Área de parâmetros
BHCJSONX	Área auxiliar JSON

41. Uso de Subprogramas

O uso de subprogramas é recomendado.

Sugestão:

```
BHCSH001 - Validar transação financeira
```

Esse subprograma poderá validar:

```

Cliente encontrado
Cliente ativo
Tipo de transação
Valor da transação
Saldo suficiente

```

Pode receber parâmetros via LINKAGE SECTION.

Códigos de retorno sugeridos:

Código	Descrição
00	Transação válida
01	Cliente não encontrado
02	Cliente inativo
03	Cliente bloqueado
04	Tipo inválido
05	Valor inválido
06	Saldo insuficiente
99	Erro técnico

42. Critérios de Aceite

O projeto será aceito quando:

1. BHCPH000 compilar sem erro.
2. BHCPH001 compilar sem erro.
3. BHCPH000 gerar BHCFHCLI com 20 registros.
4. BHCPH000 gerar BHCFHTRA com 40 registros.
5. BHCPH001 ler BHCFHCLI corretamente.
6. BHCPH001 ler BHCFHTRA corretamente.
7. BHCPH001 carregar clientes em tabela interna.
8. BHCPH001 validar transações corretamente.
9. BHCPH001 atualizar saldos corretamente.
10. BHCPH001 gerar extrato.
11. BHCPH001 gerar log.
12. BHCPH001 gerar JSON.
13. Ambos exibirem totalizadores.
14. Ambos tratarem FILE STATUS.
15. Ambos seguirem o padrão visual definido.
16. Ambos finalizarem com GOBACK.

43. Entregáveis

Cada participante deverá entregar:

1. Fonte do programa BHCPH000.
2. Fonte do programa BHCPH001.
3. Fonte do programa BHCSH001.
4. Fonte dos subprogramas, se houver.
5. Copybooks, se houver.
6. Arquivo BHCFHCLI gerado pelo BHCPH000.
7. Arquivo BHCFHTRA gerado pelo BHCPH000.
8. Arquivo BHCFHEXT gerado pelo BHCPH001.

9. Arquivo BHCFHLOG gerado pelo BHCPH001.
10. Arquivo BHCFHJSN gerado pelo BHCPH001.
11. Evidência da SYSOUT do BHCPH000.
12. Evidência da SYSOUT do BHCPH001.
13. Breve explicação da solução.